LAB 08 — Pretrained Transformers and BERT Fine-Tuning

Môn học: Xử lý ngôn ngữ tự nhiên và ứng dụng | **Học kỳ:** Học kỳ I - 2026

Giảng viên / TA: Phạm Ngọc Hải ▪ **Hạn nộp:** 23:59 - 11/11/2026

Chủ đề: From Training a Transformer to Using a Pretrained Language Model

---

1. Vai trò của LAB 08 trong toàn bộ học phần

LAB 07 đã xây dựng Transformer từ các thành phần:

```
Self-Attention
      ↓
Multi-Head Attention
      ↓
Positional Encoding
      ↓
Feed-Forward
      ↓
Residual + LayerNorm
      ↓
Transformer Encoder
```

Tuy nhiên, một Transformer được xây dựng từ đầu vẫn phải học các biểu diễn ngôn ngữ từ dữ liệu huấn luyện của chính bài toán.

LAB 08 đặt câu hỏi:

> Nếu một Transformer đã được huấn luyện trước trên một lượng văn bản rất lớn thì có thể sử dụng kiến thức đó cho một bài toán mới như thế nào?

Mạch kiến thức:

```
LAB 07
Transformer trained from scratch
        ↓
LAB 08
Pretraining
        ↓
Masked Language Modeling
        ↓
Pretrained Transformer
        ↓
Tokenizer + Contextual Embedding
        ↓
Fine-tuning
        ↓
Text Classification
        ↓
LAB 09
NER / Information Extraction
```

---

2. Learning Outcomes

Sau LAB 08, sinh viên có thể:

1. Giải thích sự khác nhau giữa **pretraining** và **fine-tuning**.
2. Mô tả ý tưởng của **Masked Language Modeling (MLM)**.

3. Giải thích vai trò của:
   - tokenizer,
   - subword token,
   - special tokens,
   - attention mask.
4. Giải thích vai trò của `[CLS]` trong BERT-style classification.
5. Tính loss đơn giản của MLM.
6. Phân biệt:
   - training from scratch,
   - feature extraction,
   - fine-tuning.
7. Sử dụng một pretrained Transformer cho classification.
8. So sánh:
   - Transformer train from scratch,
   - pretrained Transformer + fine-tuning.
9. Thực hiện thí nghiệm freezing và fine-tuning.
10. Phân tích lỗi của pretrained model.
11. Nhận diện các hạn chế:
    - domain shift,
    - tokenizer limitations,
    - sequence length,
    - computational cost,
    - catastrophic forgetting.
12. Giải thích tại sao pretrained Transformer trở thành nền tảng cho nhiều NLP applications hiện đại.

---

3. Kiến thức nền

Sinh viên phải sử dụng lại kiến thức từ LAB 06–07:

- attention;
- self-attention;
- multi-head attention;
- Transformer Encoder;
- residual connection;
- LayerNorm;
- positional information.

Không dành phần lớn thời gian để dạy lại Transformer.

---

4. Pretraining vs Fine-tuning

4.1. Training from scratch

Ở LAB 07:

$$D_{\text{task}} \rightarrow Transformer \rightarrow \text{task loss}$$

Ví dụ:

```
IMDb
 ↓
Transformer
 ↓
Sentiment classifier
```

Model phải học representation và task prediction đồng thời.

4.2. Pretraining

Một model được huấn luyện trước trên lượng văn bản lớn:

$$D_{\text{large corpus}} \rightarrow \text{Transformer} \rightarrow \text{language representation}$$

Mục tiêu không nhất thiết là classification.

Ví dụ với Masked Language Modeling:

```
The cat [MASK] fish.
          ↓
        eats
```

Model học cách dự đoán token bị che.

4.3. Fine-tuning

Sau pretraining:

$$\theta_{\text{pretrained}} \rightarrow \theta_{\text{task}}$$

Model pretrained được tiếp tục huấn luyện trên dataset cụ thể.

Ví dụ:

```
BERT
 ↓
IMDb
 ↓
Sentiment classification
```

Điểm quan trọng:

> Fine-tuning không bắt đầu từ một model có trọng số ngẫu nhiên.

---

5. Subword Tokenization

Một từ không nhất thiết tương ứng với một token.

Ví dụ minh họa:

```
unbelievable
 ↓
un + believe + able
```

hoặc một tokenizer cụ thể có thể tạo ra các subword khác.

Sinh viên cần hiểu:

```
raw text
   ↓
tokenizer
   ↓
token IDs
   ↓
embedding
   ↓
Transformer
```

---

6. Special Tokens

Với BERT-style models, sinh viên cần nhận diện vai trò của các token đặc biệt.

Ví dụ:

```
[CLS] this movie is great [SEP]
```

và:

```
[CLS] sentence A [SEP] sentence B [SEP]
```

Các token thường gặp:

Token	Vai trò
`[CLS]`	đại diện cho toàn bộ sequence trong nhiều classification setup
`[SEP]`	phân tách sequence
`[PAD]`	padding
`[MASK]`	token bị che trong MLM
`[UNK]`	token không nhận diện được, tùy tokenizer/model

Không yêu cầu sinh viên ghi nhớ máy móc tên token của mọi pretrained model.

---

7. Attention Mask

Cho sequence:

```
[CLS] the cat eats fish [PAD] [PAD]
```

attention mask có thể biểu diễn:

$$[1, 1, 1, 1, 1, 0, 0]$$

Trong đó:

- `1`: token hợp lệ;
- `0`: padding.

Mục đích:

> Không để padding token tham gia như nội dung thực của sequence.

---

8. Masked Language Modeling

Cho câu:

```
The cat eats fish.
```

Sau masking:

```
The cat [MASK] fish.
```

Model phải dự đoán:

$$P(w_t \mid w_{\neq t})$$

với (w_t) là token bị mask.

Loss cross-entropy:

$$L = -\sum_{i \in M} \log P(y_i|x)$$

Trong đó:

- (M): tập vị trí bị mask;
- (y_i): token thật;
- (P(y_i|x)): xác suất model dự đoán token đó.

---

9. Calculation Exercise 1 — MLM Loss

Giả sử chỉ có một token bị mask.

Model dự đoán:

```
cat     0.10
dog     0.05
eats    0.70
fish    0.15
```

Token đúng là:

```
eats
```

Sinh viên tính:

$$L = -\log(0.70)$$

Sau đó trả lời:

1. Nếu model dự đoán `eats = 0.95`, loss thay đổi như thế nào?
2. Nếu model dự đoán `eats = 0.01`, loss thay đổi như thế nào?
3. Vì sao cross-entropy phù hợp với bài toán MLM?

---

10. Calculation Exercise 2 — Tokenization

Cho:

```
"The cats are playing."
```

Sinh viên:

1. Tokenize bằng tokenizer được cung cấp.
2. Ghi token sequence.
3. Ghi token IDs.
4. Xác định special tokens.
5. Xác định attention mask.
6. So sánh số lượng:
   - words,
   - tokens,
   - subword tokens.

Không yêu cầu sinh viên tự đoán tokenization của tokenizer trước khi chạy.

---

11. Prediction Before Experiment

Trước khi chạy pretrained model, sinh viên phải dự đoán:

Prediction 1

Một pretrained Transformer fine-tuned trên IMDb có khả năng đạt kết quả tốt hơn Transformer nhỏ được train from scratch trên cùng dữ liệu không?

Sinh viên phải ghi:

```
Prediction:
Reason:
Expected evidence:
```

Prediction 2

Nếu freeze toàn bộ pretrained encoder và chỉ train classification head:

```
BERT encoder
    ↓ frozen
classification head
    ↓ train
prediction
```

kết quả sẽ:

```
A. tốt hơn fine-tuning toàn bộ
B. tương đương
C. kém hơn
D. không thể dự đoán
```

Sinh viên phải giải thích trước khi chạy.

Prediction 3

Nếu chỉ có 10% training data:

```
100%
50%
25%
10%
```

pretrained model hay model train from scratch sẽ giảm performance mạnh hơn?

Không được trả lời chỉ bằng trực giác; phải đưa ra cơ sở từ vai trò của pretraining.

---

12. Core Implementation

12.1. Tokenization

Sinh viên sử dụng tokenizer của pretrained model.

Ví dụ:

```
tokens = tokenizer(
    text,
    padding=True,
    truncation=True,
    return_tensors="pt"
)
```

Sinh viên phải kiểm tra:

```
tokens.keys()
```

và giải thích:

```
input_ids
attention_mask
```

---

13. Core Implementation — Classification Head

Pipeline:

```
Input text
     ↓
Tokenizer
     ↓
Input IDs
     ↓
Pretrained Transformer
     ↓
Contextual representations
     ↓
[CLS] representation
     ↓
Linear layer
     ↓
Class logits
     ↓
Softmax
```

```
    ↓
Prediction
```

Sinh viên phải hiểu rằng classification head là thành phần task-specific.

---

14. Experiment 1 — Pretrained vs From Scratch

Sử dụng cùng:

- dataset;
- train/validation/test split;
- evaluation metric;
- maximum sequence length;
- số epoch nếu hợp lý.

So sánh:

Model A

```
Transformer
random initialization
        ↓
training
        ↓
classification
```

Model B

```
pretrained Transformer
        ↓
fine-tuning
        ↓
classification
```

Kết quả:

Model	Accuracy	F1	Training time	# Parameters
Transformer from scratch				
Pretrained Transformer				

Sinh viên không chỉ báo cáo accuracy.

Phải trả lời:

1. Representation khác nhau như thế nào?
2. Training data cần thiết khác nhau như thế nào?
3. Chi phí tính toán khác nhau như thế nào?
4. Kết quả có phù hợp với prediction không?

---

15. Experiment 2 — Feature Extraction vs Fine-Tuning

So sánh hai chế độ.

Mode A — Frozen

```
Pretrained Transformer
        │
      frozen
        ↓
classification head
        ↓
prediction
```

Mode B — Fine-tuning

```
Pretrained Transformer
        │
   trainable
        ↓
classification head
        ↓
prediction
```

Báo cáo:

Mode	Encoder trainable?	Accuracy	F1	Training time
Frozen	No			
Fine-tuning	Yes			

Sinh viên phải giải thích:

Vì sao frozen encoder vẫn có thể thực hiện classification?

và:

Vì sao fine-tuning có thể cải thiện kết quả?

---

16. Experiment 3 — Amount of Training Data

Tạo các training subsets:

```
10%
25%
50%
100%
```

Giữ validation/test cố định.

So sánh:

```
training data
      ↓
10% ─┐
25% ─┤
50% ─┤ → pretrained model
100% ─┘
```

Báo cáo:

Training data	Accuracy	F1
10%		
25%		
50%		
100%		

Vẽ learning/data-efficiency curve.

Câu hỏi:

Pretraining có giúp model sử dụng dữ liệu downstream hiệu quả hơn không?

Không được kết luận chỉ từ một con số; phải dựa trên toàn bộ đường cong.

---

17. Experiment 4 — Maximum Sequence Length

So sánh:

$$T \in \{64, 128, 256\}$$

Báo cáo:

Max length	Accuracy	F1	Training time	GPU/CPU memory
64				
128				
256				

Liên hệ với LAB 06–07:

$$O(T^2)$$

Câu hỏi:

Tăng sequence length luôn làm model tốt hơn không?

---

18. Experiment 5 — Domain Shift

Đây là phần quan trọng để tránh biến LAB 08 thành bài "chạy BERT rồi báo accuracy".

Sinh viên chọn hai tập văn bản có khác biệt về domain hoặc style.

Ví dụ:

```
Domain A → movie reviews
Domain B → news / scientific / product reviews
```

Thực hiện:

```
train on A
      ↓
test on A
      ↓
```

```
test on B
```

So sánh:

$$Performance_{in-domain}$$

và:

$$Performance_{out-of-domain}$$

Câu hỏi:

Pretrained model có nghĩa là model đã "hiểu mọi domain" không?

Sinh viên phải trả lời dựa trên kết quả.

---

19. Application — Sentiment Classification

Dataset đề xuất:

IMDb

Task:

```
review
 ↓
Pretrained Transformer
 ↓
positive / negative
```

Metrics:

- Accuracy
- Precision
- Recall
- F1

Confusion matrix bắt buộc.

Không đánh giá model chỉ bằng accuracy.

---

20. Error Analysis

Sinh viên chọn tối thiểu:

- 3 false positives;
- 3 false negatives;
- 2 examples model dự đoán rất tự tin nhưng sai.

Bảng:

Text	True	Pred	Confidence	Error type	Hypothesis

Phải phân loại lỗi nếu có thể:

```
negation
sarcasm
long-range dependency
ambiguous sentiment
domain-specific expression
rare word
tokenization
context
label ambiguity
```

Không được chỉ viết:

> Model dự đoán sai vì câu khó.

---

21. Confidence Analysis

Sinh viên lấy:

```
probabilities = softmax(logits)
confidence = probabilities.max()
```

Chia prediction thành:

```
High confidence
Medium confidence
Low confidence
```

Sau đó kiểm tra:

> Confidence cao có đồng nghĩa với prediction đúng không?

Đây là bước đầu để sinh viên nhận ra:

$$confidence \neq correctness$$

và chuẩn bị cho các nội dung đánh giá model ở các LAB sau.

---

22. Ablation / Analysis

Sinh viên chọn ít nhất **một**:

A. Frozen vs Fine-tuned

hoặc

B. 10% vs 100% training data

hoặc

C. Sequence length

hoặc

D. Pretrained vs from scratch

Không cần chạy tất cả nếu tài nguyên hạn chế, nhưng phải có ít nhất một thí nghiệm có kiểm soát.

---

23. Reproducibility

Sinh viên phải ghi:

```
Model:
Tokenizer:
Dataset:
Train split:
Validation split:
Test split:
Max sequence length:
Batch size:
Learning rate:
Epochs:
Optimizer:
Random seed:
Hardware:
Training time:
```

Mục tiêu:

> Một kết quả NLP không chỉ là một con số accuracy.

---

25. Deliverables

```
lab08/
├── README.md
├── calculations.md
├── prediction.md
├── tokenization.ipynb
├── pretrained_classification.py
├── fine_tuning.ipynb
```

```
├── results.csv
├── data_efficiency.png
├── confusion_matrix.png
├── confidence_analysis.png
├── error_analysis.md
└── reflection.md
```

---

26. Rubric

Component	Points
Theory & calculation	10
Prediction before experiment	10
Tokenization & attention mask	10
Pretrained model implementation	15
From-scratch vs pretrained	10
Frozen vs fine-tuning	10
Data-efficiency experiment	10
Quantitative evaluation	10
Error & confidence analysis	10
Reflection	5
Total	**100**

---

27. Individual Learning Check

Mỗi sinh viên trả lời ngắn:

1. Pretraining khác fine-tuning ở đâu?
2. MLM là gì?
3. Tại sao cần tokenizer?
4. Subword token giải quyết vấn đề gì?
5. `[CLS]` thường được sử dụng như thế nào?
6. Attention mask dùng để làm gì?
7. Frozen encoder nghĩa là gì?
8. Fine-tuning thay đổi những tham số nào?
9. Vì sao pretrained model có thể hữu ích khi downstream dataset nhỏ?
10. Pretrained model có đảm bảo không bị domain shift không?
11. Vì sao tăng sequence length làm chi phí attention tăng?
12. Confidence cao có đảm bảo prediction đúng không?

Sinh viên phải giải thích bằng lời của mình, không chỉ đọc code.

---

28. Error Analysis — câu hỏi bắt buộc

Sinh viên phải trả lời:

Q1

Model nào hưởng lợi nhiều hơn khi training data ít?

Q2

Nếu pretrained model tốt hơn from-scratch, điều đó chứng minh điều gì?

Q3

Nếu frozen model gần bằng fine-tuned model, điều đó có ý nghĩa gì?

Q4

Nếu fine-tuning tốt hơn frozen model rất nhiều, điều đó gợi ý điều gì?

Q5

Nếu performance giảm mạnh trên domain khác, nguyên nhân có thể là gì?

Q6

Nếu sequence length tăng nhưng performance gần như không đổi trong khi training time tăng mạnh, có nên tiếp tục tăng sequence length không?

Sinh viên phải phân biệt:

```
observation
    ↓
evidence
    ↓
interpretation
```

Không được nhảy trực tiếp từ:

```
accuracy = 91%
```

sang:

```
model hiểu ngôn ngữ tốt.
```

---

29. Reflection

Viết 300–500 từ:

> Sau LAB 07, chúng ta đã biết cách xây dựng Transformer. Sau LAB 08, tại sao trong thực tế người ta thường không bắt đầu bằng việc train Transformer từ đầu?

Bài reflection phải đề cập tối thiểu:

1. data;
2. pretrained representation;
3. computational cost;
4. downstream task;
5. fine-tuning;
6. limitation/domain shift.

---

30. Bridge to LAB 09

LAB 08 kết thúc bằng:

```
Pretrained Transformer
        ↓
Fine-tuning
        ↓
Text Classification
```

Nhưng classification chỉ tạo ra **một nhãn cho toàn bộ text**.

Câu hỏi tiếp theo:

> Nếu chúng ta muốn biết **từ nào trong câu là tên người, tổ chức, địa điểm hoặc thực thể y sinh**, thì sao?

Ví dụ:

```
Barack Obama visited Paris.
```

Thay vì:

```
→ political text
```

ta muốn:

```
Barack Obama → PERSON
Paris        → LOCATION
```

Do đó LAB 09 chuyển từ:

```
Sequence Classification
```

sang:

```
Token Classification
        ↓
Named Entity Recognition
        ↓
Information Extraction
```

và tiếp tục khai thác pretrained Transformer.